

CS149 Assignment 1

Yifan Luo

September 2, 2026

Contents

1	Program 1: Parallel Fractal Generation Using Threads	2
1.1	Mandelbrot Set with 2 Threads	2
1.2	Mandelbrot Set with More Threads	3
1.3	Measuring Workload Imbalance	4
1.4	Balanced Assignment with Round-robin	5
1.5	What About More Threads?	6
2	Program 2: Vectorizing Code Using SIMD Intrinsics	6
2.1	Implementing Clamped Exponential	6
2.2	Vector Utilization Analysis	6
2.3	Implementing Array Sum	7
3	Program 3: Parallel Fractal Generation Using ISPC	8
3.1	A Few ISPC Basics	8
3.2	ISPC Tasks	8
3.2.1	Mandelbrot Set with 2 ISPC Tasks	9
3.2.2	Mandelbrot Set with More ISPC Tasks	9
3.2.3	Difference Between Threads and ISPC Tasks	9
4	Program 4: Iterative sqrt	10
4.1	Iterative sqrt with ISPC	10
4.2	The Best-Case Input	10
4.3	The Worst-Case Input	11
4.4	Beating ISPC with AVX2 Intrinsics	11
5	Program 5: BLAS SAXPY	12
5.1	Performance Analysis	12
5.2	Why Are There 4 I/O Operations?	12
5.3	Better Performance with Streaming Stores	12
6	Program 6: Making K-Means Faster	13
6.1	Generating the Data	13
6.2	The Serial Implementation	13
6.3	Identifying the Bottleneck	13
6.4	Parallelizing the Program	14

Environment

All experiments in this assignment were conducted on my personal computer, which is equipped with an Intel Core i9-13900HX processor. The CPU has 24 physical cores and 32 logical threads, with 8 of the cores supporting Hyper-Threading. Since the experiments were not conducted on the *myth* machines provided by Stanford for CS149 students, the performance characteristics and experimental results may differ from those obtained on the official course hardware.

1 Program 1: Parallel Fractal Generation Using Threads

In this exercise, we parallelize the computation of the Mandelbrot set using `std::thread`, and investigate the effects of workload distribution on performance.

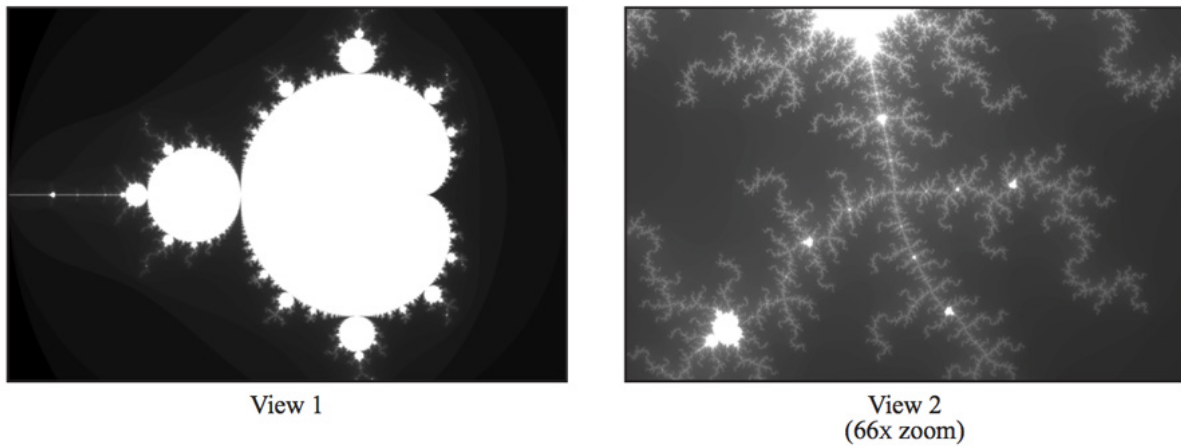


Figure 1: 2 views of Mandelbrot set.

1.1 Mandelbrot Set with 2 Threads

In this section, we use two threads to compute the Mandelbrot set. Specifically, thread 0 is responsible for computing the top half of the image, while thread 1 computes the bottom half. This work assignment is illustrated in Figure 2.

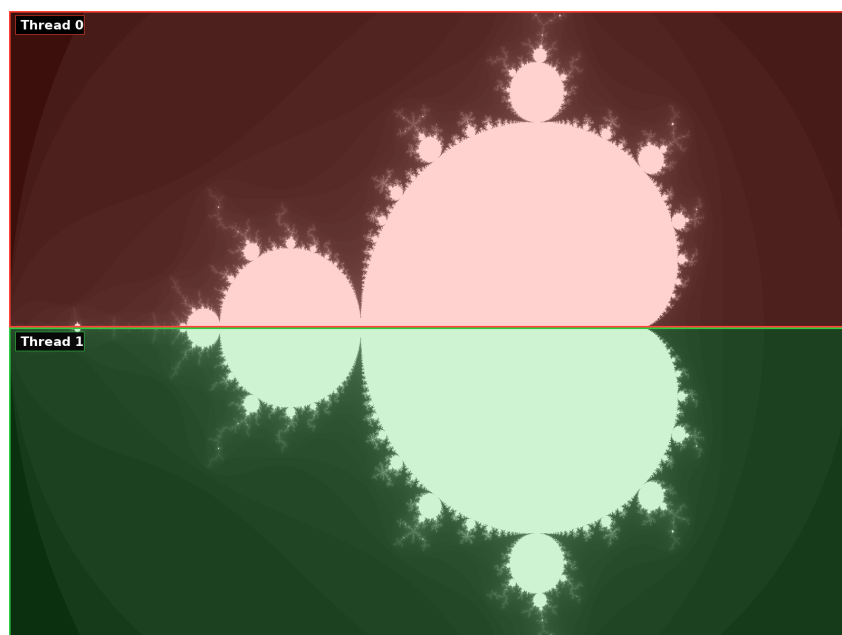


Figure 2: Work assignment with 2 threads.

With this work assignment, parallelizing the computation using two threads achieves an approximately $2\times$ speedup compared with the serial implementation. The measured execution times are shown below.

```
[mandelbrot serial]:          [419.466] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot thread]:          [210.941] ms
```

Wrote image file mandelbrot-thread.ppm
(1.99x speedup from 2 threads)

1.2 Mandelbrot Set with More Threads

In this section, we increase the number of threads from 2 to 3, 4, 5, 6, 7, and 8. The workload is partitioned in the same blocked fashion as in the previous section, as illustrated in Figure 3, which shows the 3-thread case as an example.

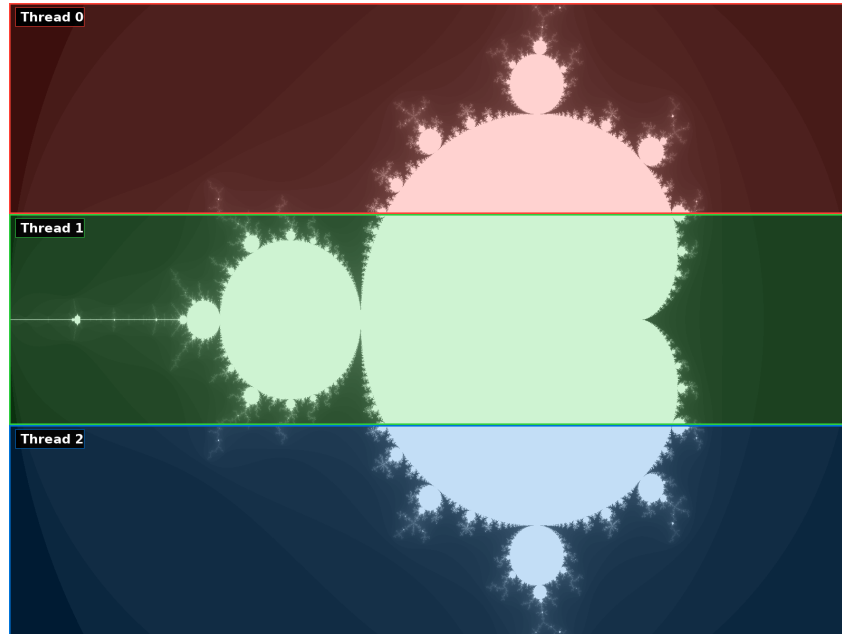


Figure 3: Work assignment with 3 threads.

The following figure shows the measured speedup as a function of the number of threads. Except for the 2-thread case, which achieves approximately a $2\times$ speedup, the measured speedup is significantly lower than the ideal linear speedup.

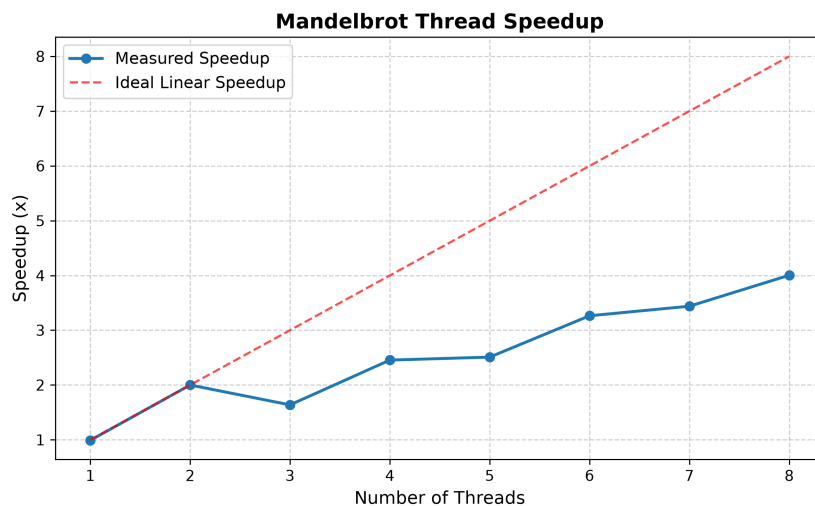


Figure 4: Speedup vs. number of threads for view 1.

In particular, the 3-threaded implementation actually performs worse than the 2-threaded version. A closer look at the work assignment in Figure 3 suggests that this performance degradation is caused by an imbalance in the workload across threads. In our visualization of the Mandelbrot set, brighter

pixels indicate longer convergence times. Thread 1 is responsible for computing the middle portion of the image, which contains a large fraction of the white pixels. As a result, thread 1 has substantially more work to perform than threads 0 and 2. The latter threads may finish their assigned work much earlier and remain idle while thread 1 continues processing its larger workload.

The same issue persists as the number of threads increases. Since the blocked partitioning does not account for the uneven computational cost of different regions of the image, the workload remains imbalanced, resulting in a speedup that falls short of the ideal linear scaling.

We also measured the speedup for view 2, as shown in Figure 5.

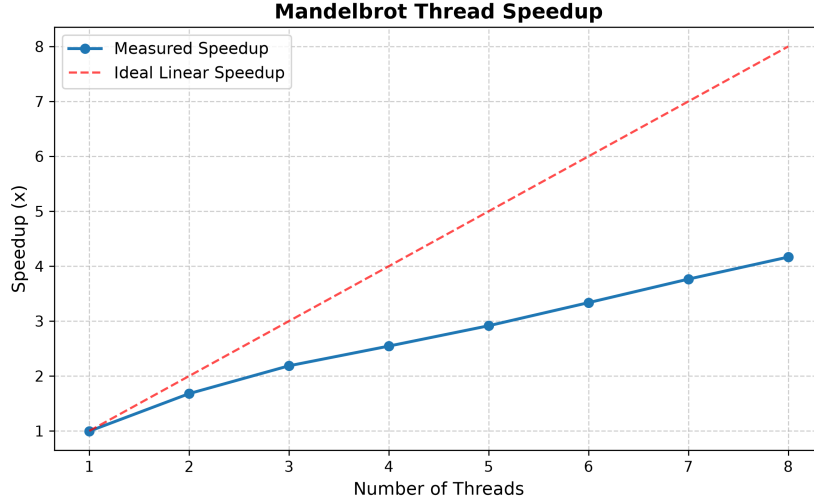


Figure 5: Speedup vs. number of threads for view 2.

In this case, even the 2-threaded implementation does not achieve a $2\times$ speedup. This is because the upper and lower halves of view 2 are not symmetric.

1.3 Measuring Workload Imbalance

To verify our hypothesis that workload imbalance is responsible for the less-than-linear speedup, we measured the execution time of each thread in the 3-thread case. The measured results are shown below.

```
[mandelbrot serial]:          [417.215] ms
Wrote image file mandelbrot-serial.ppm
[worker time(id: 0)]:        [83.113] ms
[worker time(id: 2)]:        [84.804] ms
[worker time(id: 1)]:        [256.023] ms
[worker time(id: 0)]:        [82.795] ms
[worker time(id: 2)]:        [83.171] ms
[worker time(id: 1)]:        [254.004] ms
[worker time(id: 2)]:        [82.176] ms
[worker time(id: 0)]:        [83.196] ms
[worker time(id: 1)]:        [258.036] ms
[worker time(id: 0)]:        [82.613] ms
[worker time(id: 2)]:        [84.488] ms
[worker time(id: 1)]:        [257.712] ms
[worker time(id: 0)]:        [82.579] ms
[worker time(id: 2)]:        [83.033] ms
[worker time(id: 1)]:        [258.367] ms
[mandelbrot thread]:         [254.257] ms
Wrote image file mandelbrot-thread.ppm
                                (1.64x speedup from 3 threads)
```

The measurements clearly confirm the workload imbalance. Threads 0 and 2 each take approximately 83–85 ms, whereas thread 1 consistently takes approximately 254–258 ms, which is more than three times

as long. Consequently, threads 0 and 2 finish their assigned work much earlier and remain idle while thread 1 continues processing. Since the overall execution time is determined by the slowest thread, the excessive workload assigned to thread 1 becomes the primary bottleneck and prevents the program from achieving close to linear speedup.

1.4 Balanced Assignment with Round-robin

To achieve a more balanced workload distribution, we adopt a round-robin assignment strategy. Suppose there are K threads with IDs $0, \dots, K - 1$. Thread k is responsible for rows

$$\{k, k + K, k + 2K, \dots\}.$$

The resulting work assignment is illustrated in Figure 6. In the visualization, different colors represent different threads. The round-robin assignment therefore produces an interleaved “rainbow” pattern.

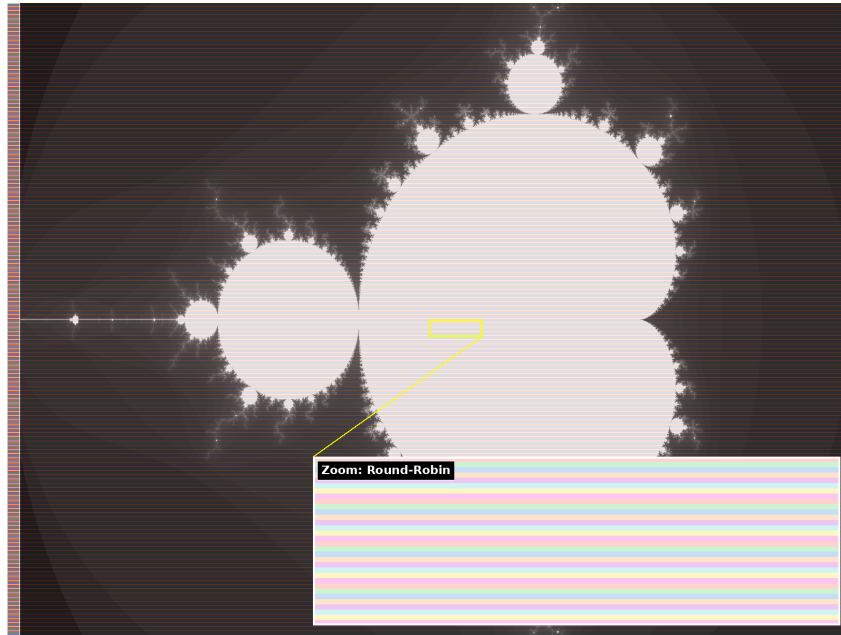


Figure 6: Round-robin work assignment.

The intuition behind this approach is straightforward. Since neighboring rows of the Mandelbrot set tend to have similar computational costs, assigning contiguous rows to the different threads results in a balanced workload.

The measured speedup for both view 1 and view 2 with 8 threads is shown below:

```
[mandelbrot serial]:          [420.976] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot thread]:         [55.584] ms
Wrote image file mandelbrot-thread.ppm
                               (7.57x speedup from 8 threads)
```

```
[mandelbrot serial]:          [235.334] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot thread]:         [31.981] ms
Wrote image file mandelbrot-thread.ppm
                               (7.36x speedup from 8 threads)
```

The resulting $7.57\times$ and $7.36\times$ speedup is close to the ideal $8\times$ linear speedup, demonstrating that the round-robin assignment substantially improves workload balance and parallel performance.

1.5 What About More Threads?

If we continue increasing the number of threads, the performance continues to improve beyond 8 threads, although the speedup eventually begins to saturate. We achieve a speedup of $12.60\times$ with 16 threads, $17.80\times$ with 24 threads, $16.64\times$ with 32 threads, and $16.73\times$ with 64 threads.

If we had conducted the experiments on Stanford’s *myth* machines, we would not expect to achieve much more than an $8\times$ speedup, since these machines have only 4 physical cores and 8 logical threads. In contrast, my personal machine has 24 physical cores and 32 logical threads, allowing the program to benefit from using more than 8 threads.

Interestingly, the performance with 32 threads is slightly worse than with 24 threads, despite the CPU supporting 32 logical threads. This suggests that the number of logical threads is not necessarily the optimal number of threads for a given workload. The additional threads may introduce increased scheduling, context-switching, and resource contention overhead, particularly because the computer is running other processes concurrently.

With 64 threads, the program also exhibits a slight performance degradation compared with 24 threads. This is expected because 64 software threads significantly exceeds the 32 logical threads supported by the CPU.

2 Program 2: Vectorizing Code Using SIMD Intrinsics

In this exercise, we use CS149’s “fake vector intrinsics” to implement a vectorized clamped exponential function and an array sum function.

2.1 Implementing Clamped Exponential

The implementation is relatively straightforward. The main consideration is handling cases where N is not a multiple of `VECTOR_WIDTH`. To correctly process the remaining elements without accessing memory beyond the end of the array, we construct a mask for the final vector iteration. The mask is then applied to the subsequent vector operations, as shown below.

```
1 int actual_vector_width = std::min(VECTOR_WIDTH, N - i);
2 __cs149_mask mask_vec = _cs149_init_ones(actual_vector_width);
```

2.2 Vector Utilization Analysis

For vector widths of 2, 4, 8, and 16, the corresponding vector utilizations are 88.5%, 86.0%, 84.9%, and 84.3%, respectively.

We observe that vector utilization decreases as the vector width increases. The primary reason is the following `while` loop:

```
1 while (all_count > 0) {
2     _cs149_vmult_float(result, result, x, mask_count_gt_zero);
3     _cs149_vsub_int(count, count, one_int, mask_vec);
4
5     _cs149_vgt_int(mask_count_gt_zero, count, zero, mask_y_is_not_zero);
6     all_count = _cs149_cntbits(mask_count_gt_zero);
7 }
```

The loop terminates only when all lanes have finished their respective computations. In other words, the execution time of the loop is determined by the lane with the largest exponent, while lanes that finish earlier become inactive and remain masked off. As the vector width increases, there is a greater chance that a lane with a relatively large exponent is included in the vector. This can cause other lanes to remain inactive for more iterations, thereby reducing the overall vector utilization.

Another possible source of lower utilization is the partially filled final vector when N is not a multiple of the vector width. A larger vector width can result in more inactive lanes in the final iteration. However, this effect is negligible in the particular setting of this exercise, since $N = 10000$ is divisible by 2, 4, 8, and 16. Therefore, no lanes are wasted due to a partially filled final vector in these experiments.

There is also an interesting observation when $s = 16$, corresponding to a workload size of 16 (the default value for `-s`). The measured vector statistics are as follows:

```

***** Printing Vector Unit Statistics *****
Vector Width:           8
Total Vector Instructions: 96
Vector Utilization:     87.0%
Utilized Vector Lanes:  668
Total Vector Lanes:     768
***** Printing Vector Unit Statistics *****
Vector Width:           16
Total Vector Instructions: 50
Vector Utilization:     87.5%
Utilized Vector Lanes:  700
Total Vector Lanes:     800

```

Interestingly, vector utilization actually increases slightly when the vector width is increased from 8 to 16.

The reason is somewhat subtle. First, we observe that

$$800 - 768 = 700 - 668 = 32.$$

Thus, although the vector width of 16 results in 32 more total vector lanes, it also results in exactly 32 additional utilized lanes. These additional utilized lanes come from four vector instructions that initialize constants:

```

1  __cs149_vec_int zero = _cs149_vset_int(0);
2  __cs149_vec_float one_float = _cs149_vset_float(1.f);
3  __cs149_vec_int one_int = _cs149_vset_int(1);
4  __cs149_vec_float clamp_value = _cs149_vset_float(9.999999f);

```

Each of these four instructions utilizes all lanes of the vector. Therefore, when the vector width increases from 8 to 16, these four instructions contribute an additional $4 \times 8 = 32$ utilized lanes.

This observation also highlights that vector utilization is affected not only by the algorithm's control flow and workload distribution, but also by the overhead of vector instructions such as constant initialization.

There's another implementation detail worth discussing. Notice that in the `while` loop above, we use

```

1  _cs149_vsub_int(count, count, one_int, mask_vec);

```

instead of

```

1  _cs149_vsub_int(count, count, one_int, mask_count_gt_zero);

```

The reason is that once a lane's `count` has reached zero, its value no longer affects the final output. Therefore, there is no need to mask the subtraction operation based on `mask_count_gt_zero`; we can simply continue decrementing `count` for all active lanes. This avoids the additional masking overhead while producing the same final result under normal input conditions.

I learned this optimization technique from the Intel ISPC User's Guide, where a similar idea can be expressed using the `unmasked` keyword. In this particular exercise, this optimization also helped achieve better vector utilization.

There is, however, a potential caveat. Repeatedly decrementing values that have already reached zero could eventually cause integer overflow and lead to incorrect behavior or even prevent the loop from terminating. Nevertheless, for the moderate input sizes used in this exercise, this is a safe and effective performance optimization.

2.3 Implementing Array Sum

To achieve a time complexity of $O(N/\text{VECTOR_WIDTH} + \log_2(\text{VECTOR_WIDTH}))$, we first accumulate the elements across all vectors lane by lane, and then reduce the resulting vector to a single scalar. The procedure is as follows:

- First, initialize a vector `sum_vec` to zero. We then load the input array in chunks of `VECTOR_WIDTH` elements and accumulate them into `sum_vec` using element-wise vector addition. After processing the entire array, each lane of `sum_vec` contains the sum of the elements that were mapped to that lane.

- Next, reduce the resulting vector to a single value. In each reduction step, we apply a `hadd` operation followed by an `interleave` operation. Repeating this process $\log_2(\text{VECTOR_WIDTH})$ times combines the partial sums from all lanes.
- Finally, after the reduction is complete, the first lane of `sum_vec` contains the sum of all elements in the input array. We extract this value as the final result.

Thus, the vectorized implementation performs $N/\text{VECTOR_WIDTH}$ vector additions followed by $\log_2(\text{VECTOR_WIDTH})$ reduction steps, resulting in the desired time complexity of $O(N/\text{VECTOR_WIDTH} + \log_2(\text{VECTOR_WIDTH}))$.

See the source code for further details.

3 Program 3: Parallel Fractal Generation Using ISPC

In this exercise, we turn to actual SIMD instructions generated by ISPC. In particular, we experiment with ISPC's `foreach` construct and tasking mechanism.

3.1 A Few ISPC Basics

Using the `foreach` construct, ISPC generates 8-wide AVX2 vector instructions. Therefore, assuming ideal utilization and negligible overhead, we would expect a speedup of approximately $8\times$. However, the measured speedup is only $3.51\times$:

```
[mandelbrot serial]:          [207.019] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]:           [59.017] ms
Wrote image file mandelbrot-ispc.ppm
                             (3.51x speedup from ISPC)
```

We hypothesize that the limited speedup is primarily caused by inefficient SIMD utilization due to masking. This effect is particularly pronounced in regions where black and white pixels meet in the Mandelbrot set. Near these boundaries, neighboring pixels can require significantly different numbers of iterations to converge. Once some lanes finish their computation, they become inactive and are masked off, while the remaining lanes continue executing. The inactive lanes therefore contribute no useful computation during these additional iterations. This divergence leads to wasted SIMD capacity and helps explain the gap between the observed $3.51\times$ speedup and the ideal $8\times$ speedup.

For view 2, the speedup is even lower:

```
[mandelbrot serial]:          [117.943] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]:           [41.026] ms
Wrote image file mandelbrot-ispc.ppm
                             (2.87x speedup from ISPC)
```

Qualitatively, this can be attributed to the structure of view 2, which contains more long, thin white regions. These regions tend to contain pixels with substantially different convergence behavior within the same SIMD vector, resulting in greater lane divergence and consequently more wasted computation than in view 1.

3.2 ISPC Tasks

Previously, the ISPC `foreach` construct only utilized SIMD instructions within a single core. With ISPC Tasks, we can distribute these SIMD computations across multiple cores, enabling both SIMD parallelism and multicore parallelism.

3.2.1 Mandelbrot Set with 2 ISPC Tasks

Similar to the 2-thread implementation in Program 1, we launch two ISPC tasks, with one task responsible for the top half of the image and the other responsible for the bottom half. The measured results are as follows:

```
[mandelbrot serial]:          [209.550] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]:           [60.173] ms
Wrote image file mandelbrot-ispc.ppm
[mandelbrot multicore ispc]:  [29.950] ms
Wrote image file mandelbrot-task-ispc.ppm
                             (3.48x speedup from ISPC)
                             (7.00x speedup from task ISPC)
```

The ISPC task implementation achieves a $7.00\times$ speedup over the serial implementation, which is approximately $2\times$ faster than the single-core ISPC implementation without tasks. This demonstrates that ISPC Tasks can effectively combine SIMD parallelism within each core with multicore parallelism across different cores.

3.2.2 Mandelbrot Set with More ISPC Tasks

In principle, using more tasks allows the workload to be distributed more evenly across the available cores. However, creating too many tasks can introduce additional scheduling and synchronization overhead, which may offset the benefits of finer-grained parallelism. Therefore, the optimal number of tasks is best determined empirically.

After experimenting with different task granularities, we chose to use

$$\frac{\text{height}}{16} = \frac{800}{16} = 50$$

tasks.

The measured performance is as follows:

```
[mandelbrot serial]:          [207.763] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]:           [59.675] ms
Wrote image file mandelbrot-ispc.ppm
[mandelbrot multicore ispc]:  [4.053] ms
Wrote image file mandelbrot-task-ispc.ppm
                             (3.48x speedup from ISPC)
                             (51.26x speedup from task ISPC)
```

With 50 ISPC tasks, the multicore implementation achieves a remarkable $51.26\times$ speedup over the serial implementation. This represents a substantial improvement over the single-core ISPC implementation and demonstrates the effectiveness of combining SIMD vectorization with fine-grained task parallelism.

3.2.3 Difference Between Threads and ISPC Tasks

There is a fundamental difference between OS threads and ISPC tasks.

A thread is an abstraction provided by the operating system that represents an independent execution context, including a program counter, registers, stack, and other associated state. To execute a thread, the operating system must schedule it onto an available hardware execution context. If the number of runnable threads exceeds the available hardware execution contexts, the OS must perform context switches to share the CPU among them. These context switches introduce nontrivial overhead, potentially costing hundreds of cycles or more. In extreme cases, additional overhead may arise from events such as TLB misses.

An ISPC task, on the other hand, is primarily an abstraction representing a unit of work provided by the ISPC runtime. Conceptually, tasks can be viewed as work items waiting in a queue. A pool of

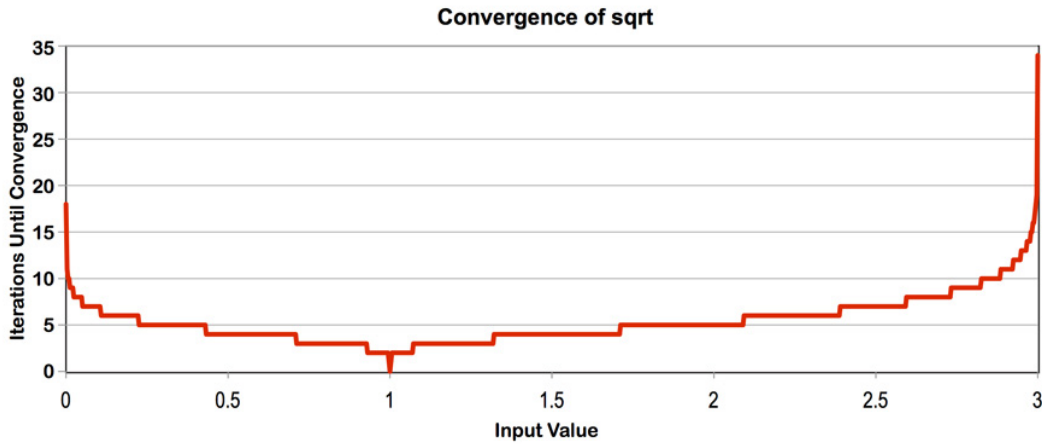
worker threads managed by the ISPC runtime can repeatedly fetch tasks from this queue and execute them. Thus, creating more tasks does not necessarily create more operating-system threads.

This distinction becomes particularly important when considering a large number of work items. For example, creating 10,000 OS threads could have severe consequences. Each thread requires its own stack and associated kernel and user-space state, resulting in substantial memory consumption and potentially even an out-of-memory (OOM) condition. Even if sufficient memory is available, the large number of runnable threads can cause excessive context switching and scheduling overhead because there are far fewer hardware execution contexts available.

Creating 10,000 ISPC tasks, in contrast, is much less problematic. The tasks can simply remain in the runtime's work queue until a worker thread becomes available. The number of tasks can therefore be much larger than the number of worker threads without requiring a corresponding increase in the number of execution contexts. This allows ISPC tasks to provide fine-grained parallelism while keeping the number of actual threads manageable.

4 Program 4: Iterative sqrt

In this program, square root of 20 million numbers is computed in parallel using Newton's method. We constructed the best and worst case input for the ISPC program, and also implemented a AVX2 intrinsics program, beating the provided ISPC program.



4.1 Iterative sqrt with ISPC

Running the provided ISPC code yields

```
[sqrt serial]:      [948.149] ms
[sqrt ispc]:        [245.929] ms
[sqrt task ispc]:    [13.030] ms
                    (3.86x speedup from ISPC)
                    (72.76x speedup from task ISPC)
```

SIMD provides $3.86\times$ of speedup, and on top of that, multi-core provide additional $72.76/3.86 \approx 18.85\times$ of speedup.

4.2 The Best-Case Input

To improve the relative speedup of the ISPC implementation, we first consider the factors limiting its performance. As discussed in the previous sections, SIMD execution may waste computational resources due to masking when different lanes require different numbers of iterations to converge. A natural approach is therefore to construct an input for which all lanes finish their computations at approximately the same time. At the same time, the input should contain sufficiently expensive computations to ensure that the serial implementation is primarily compute-bound.

Based on this reasoning, an ideal input is an array in which every element is set to 2.999. The resulting measurements are:

```

[sqrt serial]:          [1765.837] ms
[sqrt ispc]:           [395.907] ms
[sqrt task ispc]:       [23.071] ms
                        (4.46x speedup from ISPC)
                        (76.54x speedup from task ISPC)

```

As expected, this input leads to a higher SIMD speedup of $4.46\times$, compared with the previous results. The task-based implementation achieves a total speedup of $76.54\times$, which is also higher than the previous $72.76\times$. However, the additional multicore speedup provided by ISPC Tasks is

$$\frac{76.54}{4.46} \approx 17.16\times,$$

which is slightly lower than the previous $18.85\times$.

One possible explanation is that the uniform input causes different worker threads to finish their tasks at approximately the same time. Consequently, multiple threads may simultaneously attempt to acquire the next task from the work queue, increasing contention in the task scheduling mechanism. This additional synchronization and scheduling overhead could slightly reduce the effective multicore speedup.

4.3 The Worst-Case Input

To construct a worst-case input for SIMD execution, we aim to maximize the amount of computation wasted by inactive lanes. Since 2.999 requires many iterations to converge while 1 converges in only a single iteration, we can construct an input pattern consisting of one 2.999 followed by seven 1s. Within an 8-wide SIMD instruction, one lane will continue computing for a large number of iterations while the other seven lanes become inactive after only one iteration. In the worst case, this causes approximately 7/8 of the available SIMD computation to be wasted.

The measured performance is as follows:

```

[sqrt serial]:          [250.068] ms
[sqrt ispc]:           [401.037] ms
[sqrt task ispc]:       [22.873] ms
                        (0.62x speedup from ISPC)
                        (10.93x speedup from task ISPC)

```

Indeed, the ISPC implementation performs even worse than the serial implementation in this case. This is likely because the severe lane divergence largely eliminates the benefits of SIMD parallelism, while the vectorization itself still introduces some additional overhead.

4.4 Beating ISPC with AVX2 Intrinsics

Using AVX2 intrinsics, we can achieve even better performance than the single-core ISPC implementation.

```

[sqrt serial]:          [941.263] ms
[sqrt ispc]:           [249.873] ms
[sqrt task ispc]:       [12.220] ms
[sqrt AVX2]:           [177.248] ms
                        (3.77x speedup from ISPC)
                        (77.02x speedup from task ISPC)
                        (5.31x speedup from AVX2)

```

It is worth noting that the AVX2 implementation uses only a single CPU core. The fact that it outperforms the single-core ISPC implementation suggests that the lower-level control provided by AVX2 intrinsics can be beneficial for this workload. A multithreaded implementation combining AVX2 vectorization with multicore parallelism could potentially achieve even higher performance and may be able to outperform the task-based ISPC implementation as well.

5 Program 5: BLAS SAXPY

In this program, we use ISPC to implement the SAXPY operation. Unlike the compute-bound workloads in the previous programs, SAXPY is primarily memory-bound. We therefore focus on improving memory throughput rather than arithmetic throughput. By using streaming stores for the output array, we achieve an additional speedup of nearly 33%.

5.1 Performance Analysis

The measured performance is as follows:

[saxpy ispc]:	[10.528] ms	[28.307] GB/s	[3.799] GFLOPS
[saxpy task ispc]:	[4.731] ms	[62.993] GB/s	[8.455] GFLOPS
	(2.23x speedup from use of tasks)		

We observe a $2.23\times$ speedup from using ISPC tasks. This improvement is relatively modest given that we create 64 ISPC tasks on a CPU with 32 logical threads. The limited scalability is expected because SAXPY is primarily memory-bandwidth-bound. Once the available memory bandwidth is close to being saturated, adding more computational parallelism provides diminishing returns. In this regime, the memory subsystem, rather than the CPU's computational capacity, becomes the primary performance bottleneck.

5.2 Why Are There 4 I/O Operations?

Note that in `main.cpp`, we use

```
1 TOTAL_BYTES = 4 * N * sizeof(float);
```

to estimate the total memory traffic. At first glance, this may seem surprising, since the SAXPY operation appears to involve only two reads and one write for each element. Where does the fourth memory operation come from?

The reason is that a regular write to memory may require a read-for-ownership (RFO) transaction. Before modifying a cache line, the CPU may first load the corresponding cache line into the cache. The CPU then modifies the data in the cache and eventually writes the modified cache line back to memory. Thus, in addition to the two reads of the input arrays, the output array incurs both an RFO read and a write-back, resulting in a total of four memory transfers: two reads from the input arrays, one read associated with the output write, and one write-back to memory.

5.3 Better Performance with Streaming Stores

Since the output data will not be read or modified again, we can avoid the read-for-ownership (RFO) transaction by using streaming stores. This reduces the memory traffic from four transfers per element to three, so ideally we would expect a speedup of

$$\frac{4}{3} \approx 1.33 \times .$$

The measured performance with streaming stores is as follows:

[saxpy ispc]:	[7.940] ms	[37.534] GB/s	[5.038] GFLOPS
[saxpy task ispc]:	[3.717] ms	[80.188] GB/s	[10.763] GFLOPS
	(2.14x speedup from use of tasks)		

The single-core ISPC implementation achieves a speedup of

$$\frac{5.038}{3.799} \approx 1.33\times,$$

which matches the ideal speedup predicted above. This demonstrates that eliminating the unnecessary RFO traffic can significantly improve the performance of this memory-bound workload.

It is worth noting that the reported memory bandwidth values, 37.534 GB/s and 80.188 GB/s, are no longer accurate measures of the actual memory traffic. The original calculation assumes four memory

transfers per element, whereas streaming stores reduce this to three. Therefore, the actual memory bandwidth should be calculated using $3N \cdot \text{sizeof}(\text{float})$ as the total memory traffic.

There are also several implementation details worth mentioning. The `streaming_store` function provided by ISPC requires the destination address to be aligned to 32 bytes. This requirement follows from the fact that each streaming store writes

$$4 \text{ bytes/lane} \times 8 \text{ lanes} = 32 \text{ bytes}$$

at a time.

Consequently, we cannot use the standard `new` operator for memory allocation, since the memory it provides is only guaranteed to be 16-byte aligned on our platform. Instead, the arrays must be allocated with an allocator that provides 32-byte alignment. In addition, the number of elements processed by each task must be a multiple of 8, so that every streaming store operates on a properly aligned 32-byte block.

Violating either of these alignment requirements can result in a segmentation fault when executing the streaming store.

6 Program 6: Making K-Means Faster

In this exercise, we identify the performance bottleneck in the given K-Means implementation and parallelize the corresponding function to achieve a speedup of more than $2.1\times$.

6.1 Generating the Data

Since we do not have access to Stanford's servers, we use the data generation code provided in `main.cpp` to generate a synthetic dataset locally.

6.2 The Serial Implementation

After building the starter code and running the provided visualization script, we obtain the following results.

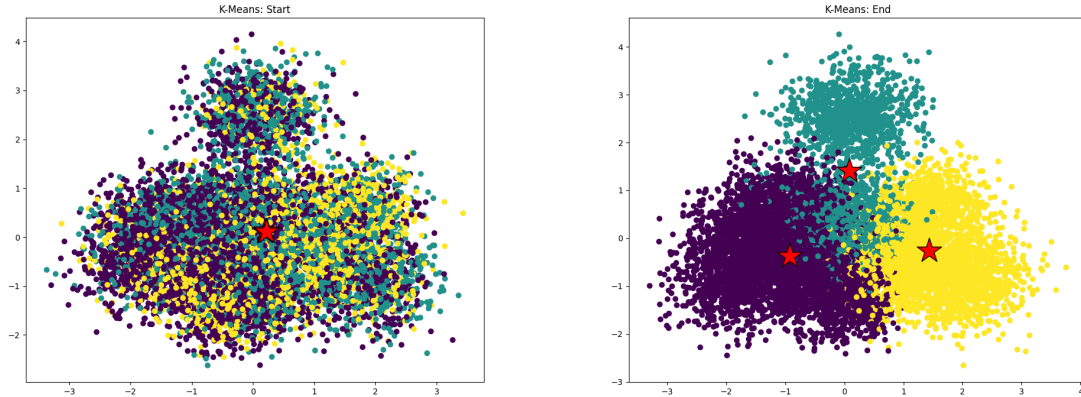


Figure 7: Results of the starter K-means implementation.

It can be observed that not all points appear to be assigned to their nearest centroid. This is because the visualization is a 2-D projection of the original high-dimensional data onto the two principal components with the largest variance. The clustering itself is performed in the original high-dimensional space, so the assignments may not appear consistent with the distances observed in the 2-D projection.

6.3 Identifying the Bottleneck

We first placed timers around `computeAssignments`, `computeCentroids`, and `computeCost` to measure the execution time of each component. The resulting measurements are shown below:

```
Running K-means with: M=1000000, N=100, K=3, epsilon=0.100000
[Compute Assignment Time]: 5186.842 ms
[Compute Centriod Time]: 1108.371 ms
[Compute Cost Time]: 1526.203 ms
[Total Time]: 7821.477 ms
```

These results show that `computeAssignments` is the main bottleneck, accounting for approximately 66% of the total execution time.

We also noticed that `dist` is called by both `computeAssignments` and `computeCost`. Therefore, we additionally measured the execution time of `dist`:

```
Running K-means with: M=1000000, N=100, K=3, epsilon=0.100000
[Compute Assignment Time]: 10321.791 ms
[Compute Centriod Time]: 1060.596 ms
[Compute Cost Time]: 3436.225 ms
[Dist Time]: 10742.353 ms
[Total Time]: 14818.667 ms
```

The results indicate that `dist` accounts for a substantial portion of the execution time in both `computeAssignments` and `computeCost`. We also note that measuring `dist` directly introduces additional overhead because it is called a very large number of times. Consequently, the measured total execution time increases significantly when timing is added to `dist`.

Although `dist` itself is computationally expensive, we ultimately chose to parallelize `computeAssignments` rather than `dist`. The reasons for this choice will be discussed in the following section.

6.4 Parallelizing the Program

Since `computeAssignments` accounts for a substantial portion of the total execution time, parallelizing this function is a natural approach to improving performance. There are two possible ways to introduce parallelism in `computeAssignments`: parallelizing across data points or parallelizing across clusters.

For the following two reasons, we believe that parallelizing across data points is the better approach:

- There are far more data points than clusters. With $M = 10^6$ data points and $K = 3$ clusters, parallelizing across clusters can provide at most a $3\times$ speedup, even without considering synchronization overhead or workload imbalance. In contrast, the much larger number of data points provides substantially more opportunities for parallelism and allows the workload to be distributed more flexibly among workers.
- Parallelizing across data points requires significantly less synchronization. The assignment of different data points to clusters is completely independent, so each worker can process a subset of the data points independently. In contrast, although the distance computations between a data point and the different centroids are independent, determining the closest centroid requires comparing these distances, which introduces a sequential dependency within each data point.

Another possibility is to parallelize `dist` itself. We believe that this would be better suited to a SIMD-based approach similar to the array-sum implementation discussed in the previous sections, using vector operations such as `hadd` and `interleave`. However, this exercise specifically asks us to use threads to parallelize the program. Since the dimensionality is only $N = 100$ and `dist` is called a very large number of times, the overhead of repeatedly creating and joining threads could outweigh the benefits of parallelizing such a small computation.

Therefore, after considering the available parallelism and the associated overhead, we chose to parallelize `computeAssignments` across data points. The resulting performance is shown below:

```
./kmeans_serial
Running K-means with: M=1000000, N=100, K=3, epsilon=0.100000
[Total Time]: 7606.635 ms
```

```
./kmeans_thread
Running K-means with: M=1000000, N=100, K=3, epsilon=0.100000
[Total Time]: 3568.560 ms
```

This parallel implementation produces the same results as the serial implementation while achieving a speedup of approximately $2.13\times$.